



DOCUMENTO 03

Requisitos No Funcionales

Usabilidad, seguridad, rendimiento, disponibilidad, compatibilidad, mantenibilidad y escalabilidad

Proyecto	PARQUEADERO — Sistema de gestión de parqueadero
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Versión / Fecha	1.0 — 1 de octubre de 2026

Requisitos No Funcionales

Alcance de este documento y fuentes de información

Este documento distingue tres tipos de información:

- EXISTENTE** Elementos verificados en el repositorio del proyecto a la fecha de elaboración.
- PROPUESTO** Diseño planteado para el sistema de parqueadero, aún no implementado en el código.
- PENDIENTE** Datos que deben ser completados por el equipo del proyecto.

El video de referencia corresponde a una sesión de clase que muestra la **metodología y las herramientas** de desarrollo (repositorio en GitHub, frontend PWA con Vite, base de datos MySQL en Clever Cloud y despliegue en Vercel). El video **no muestra funcionalidades del sistema de parqueadero**; por esta razón, las funcionalidades descritas se presentan como diseño propuesto.

Se presentan los requisitos no funcionales del sistema indicando, para cada uno, si está **respaldado por la configuración existente** del proyecto o si es una meta **propuesta**.

Código	Categoría	Requisito	Estado	Respaldo
RNF-01	Usabilidad	La interfaz debe ser adaptable (responsive) para usarse en computador y en teléfono móvil.	PROPUESTO	La plantilla actual incluye <i>viewport</i> móvil en index.html.
RNF-02	Usabilidad	Las acciones principales (ingreso y salida) deben completarse en máximo tres pasos desde el panel del vigilante.	PROPUESTO	—
RNF-03	Usabilidad	La aplicación debe poder instalarse en el dispositivo como PWA, con nombre e íconos propios.	EXISTENTE	vite-plugin-pwa configurado con manifest "Parkinlock" e íconos de 64, 192 y 512 px y maskable.
RNF-04	Seguridad	Las contraseñas deben almacenarse cifradas con un algoritmo de hash (por ejemplo, bcrypt), nunca en texto plano.	PROPUESTO	—
RNF-05	Seguridad	Las funciones deben protegerse según el rol del usuario autenticado.	PROPUESTO	—
RNF-06	Seguridad	Las credenciales de la base de datos deben manejarse en variables de entorno y no subirse al repositorio.	PROPUESTO	El .gitignore del repositorio ya excluye archivos .env.

Código	Categoría	Requisito	Estado	Respaldo
RNF-07	Rendimiento	Los recursos estáticos del frontend deben almacenarse en caché mediante service worker para acelerar las cargas posteriores.	EXISTENTE	El build genera sw.js con Workbox (modo generateSW, actualización automática).
RNF-08	Rendimiento	Las consultas de disponibilidad deben responder en menos de 2 segundos en condiciones normales.	PROPUESTO	—
RNF-09	Disponibilidad	El frontend debe desplegarse en un servicio de hosting en la nube (Vercel, según la metodología del curso).	PROPUESTO	Metodología mostrada en el video de referencia.
RNF-10	Disponibilidad	La base de datos MySQL debe alojarse en un servicio administrado en la nube (Clever Cloud, según la metodología del curso).	PROPUESTO	Metodología mostrada en el video de referencia.
RNF-11	Compatibilidad	La aplicación debe funcionar en las versiones actuales de navegadores Chrome, Edge y Firefox.	PROPUESTO	—
RNF-12	Mantenibilidad	El código fuente debe versionarse en GitHub, separando frontend y backend en carpetas independientes.	EXISTENTE	Repositorio con carpetas frontend-Parkinlock- y backend-Parkinlock.
RNF-13	Mantenibilidad	El código del frontend debe analizarse con un linter.	EXISTENTE	oxlint configurado (script npm run lint).
RNF-14	Escalabilidad	El modelo de datos debe permitir agregar nuevas zonas, espacios y tipos de vehículo sin modificar la estructura de las tablas.	PROPUESTO	Ver modelo entidad-relación propuesto.